PAGE NO.....
DATE / /

Classes and Objects

(a) Class

→ A class is a blueprint or template for creating objects.

→ It defines the attributes (data) and methods (functions) the object will have.

→ Example:

```
class Dog:
    def __init__(self, name, age)
        self.name = name  # Attribute
        self.age = age    # Attribute

    def bark(self):       # method
        print(f"{self.name} says woof!")
```

(1) Constructors

→ A constructor is a special method that is automatically called when an object of a class is created.

→ Its primary purpose is to initialise the objects attributes and setup its initial state.

→ In Python, the constructor is defined using the `__init__` method.

2) Type of Constructors

(a) Default Constructor

→ If you don't define a constructor in a class, Python provides a default constructor that does nothing.

- Example

```
class Dog:
    pass # No constructor defined.
dog = Dog() # Default constructor is called.
```

PAGE NO.....
DATE / /

(b) Parameterized Constructor

→ A parameterized constructor accepts parameters to initialize the object's attributes.

→ Example:

```
class Dog:
    def __init__(self, name, age):
        self.name = name  # Initialize attributes
        self.age = age

dog = Dog("Buddy", 3)  # Parameterized constructor is called.
```

(c) Non-Parameterized Constructor

→ A non-parameterized constructor does not accept any parameters (other than self).

→ Example:

```
class Dog:
    def __init__(self):
        self.name = "Unknown"  # Default Values
        self.age = 1

dog = Dog()  # Non-parameterized constructor is called
```

(3) Key Concepts of Constructors:

(a) self Parameter

→ The first parameter of a constructor is always self.

→ self refers to the instance of the class (the object being created).

→ It allows you to access and modify the object's attributes.

~~Class~~

(b) Initialization

→ Constructors are used to set up the initial state of an object.

→ You can assign values to attributes, initialize variables, or perform any setup tasks.

(c) Custom Arguments.

→ You can define additional parameters in the constructor to customize how objects are created.

Example

```
Class Dog:
    def __init__(self, name, age, breed):
        self.name = name
        self.age = age
        self.breed = breed.
```

(d) Default Values.

→ You can provide default values for parameters in the constructor.

→ Example:

```
Class Dog:
    def __init__(self, name, age=1): #
        self.name = name
        self.age = age
```

Default age is 1.

PAGE NO.....
DATE / /

4) Advanced Constructor Concepts.

(a) Constructor Overloading

→ Python doesn't not support constructor overloading (defining multiple constructors with different parameters) directly.

→ However, you can achieve similar functionality using default arguments or *args and **kwargs

→ Example:

```
Class Dog:
    def __init__(self, name=None, age=None):
        self.name = name
        self.age = age

dog1 = Dog()              # No arguments
dog2 = Dog("Buddy")       # one arguments.
dog3 = Dog("Max", 5)      # Two arguments.
```

(b) Dynamic Attributes.

→ You can dynamically add attributes in the constructor using setattr()

→ Example:

```
Class Dog:
    def __init__(self, **kwargs):
        for key, value in kwargs.items():
            setattr(self, key, value)

dog = Dog(name="Buddy", age=3, breed="Golden Retriever")
```

print(dog.name) # output : Buddy

(4) Calling Other Methods

→ You can call other methods from the constructor to perform additional setup

- Example

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
        self.bark() # call the bark method during
                    initialization

    def bark(self):
        print(f"{self.name} says woof!")

dog = Dog("Buddy", 3) # output : Buddy says woof!
```

(5) Destructors.

→ A destructor is a special method called when an object is about to be destroyed.

→ In Python, the destructor is defined using the `__del__` method.

→ Example.

```
Class Dog:
    def __init__(self, name):
        self.name = name

    def __del__(self):
        print(f"{self.name} is being destroyed")

dog = Dog("Buddy")
del dog # Destructor is called.
```

PAGE NO.....
DATE / /

(6) Constructor Chaining

-> constructor chaining refers to calling one constructor from another.

-> In Python, this can be achieved using the super() function to call the parent class's constructor.

-> Example:

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, age):
        super().__init__(name)  # call parent class's __init__
        self.age = age

dog = Dog("Buddy", 3)
print(dog.name)  # output: Buddy
print(dog.age)  # output: 3
```

-> The super() function is used to call a method from the parent class

-> It is commonly used in the child class's __init__ method to initialize the parent class's attributes.

PAGE NO.....
DATE / /

Object

→ An object is an instance of a class.

→ You can create multiple objects from the same class, each with its own data.

→ Example

```
dog1 = Dog("Buddy", 3)  # Create an object.
dog2 = Dog("Max", 5)  # Create another object.

dog1.bark()  # Output: Buddy says woof!
dog2.bark()  # Output: Max says woof!
```

Inheritance

→ Inheritance allows a child class to ~~her~~ inherit attributes and methods from a parent class.

→ The child class class can extend or modify the behavior of the parent class

→ Example

```
class Animal:  # Parent class.
    def speak(self):
        print("Animal speaks!")

class Dog(Animal):
    def bark(self):
        print("Dog barks.").

dog = Dog()
dog.speak()  # ~~Inheritance~~ Inherited from Animal.
dog.bark()  # Defined in Dog.
```

allows a child to inherit all the non-private attributes and methods of the parent class.

PAGE NO.....
DATE / /

Output

Animal speaks
Dog barks.

2. Types of inheritance.

(a) Single inheritance.

-> A child class inherits from one parent class.

```
class Parent:
    pass

class Child(Parent):
    pass
```

(b) Multiple Inheritance.

-> A child class inherits from multiple parent classes

-> Example.

```
class Father:
    pass

class Mother:
    pass

class Child(Father, Mother):
    pass.
```

PAGE NO.....
DATE / /

(c) Multilevel Inheritance

→ A child class inherits from a parent class, which in turn inherits from another parent class.

→ Example

```
class Grandparent:
    pass

class Parent(Grandparent):
    pass.

class Child(Parent):
    pass.
```

(d) Hierarchical Inheritance

→ Multiple child classes inherit from one parent class

→ Example

```
class Animal
    pass

class Dog(Animal):
    pass

class Cat(Animal):
    pass.
```

DATE / /

(c) Hybrid Inheritance.

→ A combination of multiple and multilevel inheritance.

→ Example

```
class A:
    pass

class B(A):
    pass

class C(A):
    pass

class D(B,C): # Inherits from B and C which both inherit from A.
    pass
```

(3) Method Overriding

→ Method overriding allows a child class to provide a specific implementation of a method that is already defined in the parent class.

Example:

```
class Animal:
    def speak(self):
        print("Animal speaks.")

class Dog(Animal):
    def speak(self):
        print("Dog barks.")

dog = Dog()
dog.speak() # output: Dog barks.
```

→ Access Modifier can be apply on method or variable (attributes)

PAGE NO.....
DATE / /

4) Access Modifiers

(a) Public in Inheritance

→ Public attributes and methods are fully accessible in child classes

→ Example:

```
Class Parent:
    def __init__(self):
        self.public_var = "I am public"

Class child(Parent):
    def show(self):
        print(self.public_var) # Accessible

child = Child()
child.show() # Output: I am public.
```

(b) Protected in Inheritance (_)

→ Protected attributes and methods are accessible in child classes.

→ Example:

```
Class Parent:
    def __init__(self):
        self._protected_var = "I am protected".

Class Child(Parent):
    def show(self):
        print(self._protected_var) # Accessible
```

```
child = Child()
child.show() # output: I am protected.
```

(c) Private in Inheritance (__) / same applied with method.

→ Private attributes and methods are not accessible in child classes

→ Python uses name mangling to make private attributes harder to access outside the class.

```
Class Parent:
    def __init__(self):
        self.__private_var = "I am private"

Class Child(Parent):
    def show(self):
        print(self._Parent__private_var)
                # Accessible with mangled name

child = Child()
child.show() # output: I am private.
```

→ Python uses name mangling to make private attributes harder to access outside the class.

→ Private attributes are renamed to [_ClassName__attributes] to avoid accidental access.

Encapsulations

PAGE NO.....
DATE / /

→ Encapsulation hides the internal details of how an object works.

→ It protects the data from unauthorized access or modification.

→ It provides a clean interface for interacting with the object.

1. Private Members

→ Private members are denoted by a double underscore (__) prefix.
→ They cannot be accessed directly from outside the class.
→ Python uses name mangling to make it harder (but not impossible) to access private members.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance  # private attribute.

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
        else:
            print("Deposit amount must be positive.")

    def get_balance(self):
        return self.__balance
        # Accessing private attributes via a method.
```

PAGE NO.....
DATE / /

~~# Creating the balance using~~

Creating an object of BankAccount.
account = BankAccount(1000)

Depositing money
account.deposit(500)

Getting the balance using the public method.
print(account.get_balance()) # output: 1500

Trying to access private attributes directly (will raise an error)
print(account.__balance) # AttributeError

Accessing the private attributes using name mangling (not recommended).
print(account._BankAccount__balance) # output: 1500.

So to have get method we have to write. compulsory get_MethodName()?

→ Not necessarily. It's just a convention to make the purposes of the method clear.

* Inherited class access parent class constructors, attributes and methods.

PAGE NO.
DATE / /

2) Protected Member

→ Protected members are denoted by a single underscore (_) prefix.

→ They are accessible within the class and its subclasses.

→ This is more of a convention than a strict rule in python.

```
class Person:
    def __init__(self, name, age):
        self._name = name  # Protected
        self._age = age

    def display(self):
        print(f"Name: {self._name}, Age: {self._age}")
    def display(self):
        print(f"Name: {self._name}, Age:
              {self._age}")

class Student(Person):
    def __init__(self, name, age, roll_no):
        super().__init__(name, age)
        self.roll_no = roll_no

    def show_student_details(self):
        print(f"Roll No: {self.roll_no},
        Name: {self._name}")
                # Accessing protected
                  Attributes
```

Inheritance access parent class constructors/attributes and parent class methods.

PAGE
DATE / /

2) Protected Member

→ Protected members are denoted by a single underscore (_) prefix.

→ They are accessible within the class and its subclasses.

→ This is more of a convention than a strict rule in python.

```
class Person:
    def __init__(self, name, age):
        self._name = name  # Protected
        self._age = age

    def display(self):
        print(f"Name: {self._name}, Age: {self._age}")
def display(self):
    print(f"Name: {self._name}, Age:
          {self._age}")

class Student(Person):
    def __init__(self, name, age, roll_no):
        super().__init__(name, age)
        self.roll_no = roll_no

    def show_student_details(self):
        print(f"Roll No: {self.roll_no},
        Name: {self._name}")
                # Accessing protected
                  Attributes
```

PAGE NO.....
DATE / /

```
student = Student("Alice", 20, 101)
student.show_student_details()
# Output: Roll No: 101, Name: Alice
```

(1) Why the display(self) method doesn't need to pass parameters like name and age. and why self is enough.?

→ self Refer to the current object. It allows you to access the object's attributes and methods.

Super Concepts

(2) What does super() Do?

→ super().__init__(name, age) calls the parent class constructor (__init__) to initialize - name and - age.

→ super() does not automatically call other methods like display.

→ If you want to call other methods from the parent class, you need to explicitly use super().methodName().

(3) Rule is same for all where the attributes or methods in the parent class are public, protected or private.

→ The purpose of super() is to invoke the parent class's behaviour not to directly access attributes.

3) Public Members

→ public members have no prefix.

→ They are accessible from anywhere.

```
Class Car:
    def __init__(self, model, year):
        self.model = model
        self.year = year

    def display(self):
        print(f"Model: {self.model},
              Year: {self.year}").

car = Car("Tesla Model S", 2023).
car.display(). #Output Model: Tesla Model S
                       Year: 2023.
print(car.model).
        #Accessing public attributes
         directly.
```

Getter and setters

PAGE NO.....
DATE / /

Encapsulation is often implemented using getter and setter method to control access to private attributes. This allow you to add validation or logic when getting or setting a value.

```
class Temperature:
    def __init__(self, celsius):
        self.__celcius = celsius  # private attributes

    @property
    def get_celsius(self):
        return self.__celsius  # Getter method

    @celsius.setter
    def set_celsius(self, value):
        if value < -273.15:
            raise ValueError("Temperature cannot be below absolute zero!")
        self.__celsius = value

temp = Temperature(25)
print(temp.get_celsius())  # output: 25.
temp.set_celsius(30)
print(temp.get_celsius())  # output: 30.

# temp.set_celsius(-200)  # Raises ValueError.
```

Feature	Getter	Setter
purpose	Retrieves the value of an attributes	Modifies the value of an attribute
Access	Read-only access	Write-only access.
Logic	Can add logic for formatting or access	can add logic for validation or updates.
Example	@property method	@attribute.setter method.

Why we need super() even though inheritance provides access to the parent class's attributes and methods?

abstract classes are designed to be inherited.

PAGE NO.....
DATE / /

Abstractions

→ Abstractions is the process of hiding the complex implementation details and exposing only the essential features of an object.

hiding implementations means the internals workings of class or method are not exposed to the outside worlds

Real-life Example

→ Think of a car, the complexity of the engine is hidden from the driver.

```
from abc import ABC, abstractmethod.

# Abstract class.
class Car(ABC)
    def __init__(self, name):
        self.name = name  # Common attributes for all cars.

    @abstractmethod.
    def start_engine(self): # Abstract method with implementation
        print("starting the engine ...")
                # Common behavior for all cars.
    @abstractmethod.
    def stop_engine(self): #  "  "
        print("stopping the engine ...") #
                #  "  "  "

# Subclass 1 : ElectricCar.
class ElectricCar(car):
    def start_engine(self): # override abstract method.
        super().start_engine(). # Call the parent class's implementation.
```

2 (9)

DATE / /

→ Abstractions is the process of hiding the complex implementation details and exposing only the essential features of an object.

Hiding implementations means the internals workings of ~~a~~ ormethod are not exposed to outside worlds

Real-life Example

→ Think of a car, the complexity of the ~~c~~ is hidden from the driver.

```
from abc import ABC, abstractmethod.

# Abstract class.
Class Car(ABC)
    def __init__(self, name):
        self.name = name  # Common attribute for all cars.

    @abstractmethod.
    def start_engine(self): # Abstract method with implementation
        print("starting the engine...")
                # Common behavior for all cars.

    @abstractmethod.
    def stop_engine(self): #  "  "
        print("stopping the engine...") #
                #  "  "  "

# Subclass 1: Electric Car.
class ElectricCar(car):
    def start_engine(self): # override abstract method.
        super().start_engine(). # Call the parent class's implementation
        return f"{self.name}'s electric
```

2 9

engine started silently."

def stop_engine(self): # " "
super().stop_engine() # " "
1 return f"{self.name}'s electric engine stopped."

Subclass 2: PetrolCar(Car):

Class PetrolCar(Car):
2 b def start_engine(self): # " "
super().start_engine() # " "
return f"{self.name}'s petrol engine started with a roar."

def stop_engine(self):
super().stop_engine() # " "
return f"{self.name}'s petrol engine stopped"

Create objects
electric_car = ElectricCar("Tesla")
petrol_car = PetrolCar("Ford").

print(electric_car.start_engine()) # caller
output: Starting the engine...
Tesla's electric engine started silently.
print(petrol_car.stop_engine())
output: Stopping the engine
Ford's petrol engine stopped.

PAGE NO.....
DATE / /

The method name be the same in both the parent class (where it's declared as an abstract method) and subclasses (where its implemented) This is because the subclass is overriding the abstract method from the parent class. If there is not same, python will not recognise it as the implementation of the abstract method.

Aspect	Abstractions	Encapsulation
Purpose	Hides complexity by exposing only essential features.	Hides data by restricting access to attributes.
Focus.	what an object does	How an object protects its data.
Implementation	Achieved using abstract classes and methods	Achieved using private / protected attributes
Example	A shape class with an abstract area() method	A BankAccount class with private --balance

PAGE NO.....
DATE / /

Why we need super() even though inheritance provides access to the parent class's attributes and methods?

(1) To call the Parent class's Constructor

- ~~When~~ when you create an object of the child class the parent class's constructor (--init--) is not automatically called unless you explicitly use super() like super().--init--()

Example: without super().

```
class Parent:
    def __init__(self, name):
        self.name = name  # Parent class attributes

class Child(Parent):
    def __init__(self, name, age):
        self.age = age  # Child class
child = Child("Alice", 10)  attributes.
# Access attributes
print(child.age)  # output: 10
print(child.name)  # AttributeError:
                   child object has no
                   attribute 'name'
```

Here,

- The Parent class's constructor is not called, so the name attribute is not initialized.
- This results in an AttributeError when trying to access child.name.

PAGE NO.....
DATE / /

Example: with super().

```
class Parent:
    def __init__(self, name):
        self.name = name

class child(Parent):
    def __init__(self, name, age):
#Call parent class's constructor super().__init__(name).
# child class attributes. self.age = age

#Create object.
child = child("Alice", 10).
# Access Attributes
print(child.name) # output: Alice.
print(child.age) # output: 10.
```

Here,

- super().__init__(name) calls the Parent class's constructor, initializing the name attribute.
- The child class's constructor initialize the age attribute.

PAGE NO.....
DATE / /

(2) To Extend Parent class's Methods.

→ If you override a method in the childclass, the parent class's version of the method is not automatically called.

→ You can use super() to call the parent class's method and extend it's behaviour

Example.

```
class Parent:
    def great(self):
        return "Hello from Parent!"

class Child(Parent):
    def great(self):
        parent_greeting = super().great()
                # call parent class's
                  method.
        return f"{parent_greeting} And.
                Hello from child"
# Create object.
child = Child()

# Call method.
print(child.great())
# output: Hello from Parent! And hello from
          Child!.
```

Here,

- The child class override the great() method.
- super().great() calls the Parent class's great() method and the child class extends its behaviour.

PAGE NO.....
DATE / /

(3) To Handle Multiple Inheritance

→ In cases of multiple inheritance (where a class inherits from more than one parent class) super() ensures that the correct parent class's method is called in the method resolution order (MRO)

↳ if2 comes in further will be discussed

```
class A:
    def __init__(self):
        print("A's constructor").

class B(A):
    def __init__(self):
        super().__init__()
        print("B's constructor").

class C(A):
    def __init__(self):
        super().__init__().
        print("C's constructor").

class D(B,C):
    def __init__(self):
        super().__init__()
        print("D's constructor").
# Create object.
```

A's constructor
C's constructor
B's constructor
D's constructor

create object.

d = D().

output

A's constructor.

C's constructor

B's constructor.

D's constructor.

abstraction (1) super() vs return

why return is used here?

-) ~~In the start_engine() method of the PetrolCar class:~~

- super().start_engine():
  - This calls the parent class's start_engine() method, which prints "Starting the engine ...."
  - It ensures that the common behaviour (printing) is executed.
- return f'{self.name}'s petrol engine started with a roar."
  - This return a custom message specific to the PetrolCar class.
  - It uses the self.name attribute, which is inherited from the parent class.

PAGE NO.....
DATE / /

Why Don't You Need super() to Access self.name?

1. Inheritance

↳ In abstractions, it automatically gets access to all the non-private attributes of the parent class.

↳ self.name is initialized by the parent class's constructor (car.__init__) and is directly accessible in the child class.

2) Automatic Constructor Call

→ When you create an object of the child class (ElectricCar or PetrolCar), the parent class's constructor (Car.__init__) is automatically called (unless you override it in the child class).

→ This initializes the self.name attribute, making it available in the child class.

3) Accessing Attributes

→ Once the self.name attribute is initialized, you can access it directly in the child class using self.name

→ You don't need to use super() to access attributes because they are part of the object.

PAGE NO.....
DATE / /

When Do you Need super()?

→ You override the constructor (__init__) in the child class and want to call the parent class's constructor.

→ You want to call a method from the parent class.

Where does return statement return value?

what does return Do?

→ The return statement is used to send a value back to the caller of the method. When a method is called, it executes its code and can optionally return a value to the caller.

How does return work Here?

2 (a) (1) super().start_engine()

- This calls the parent class's start_engine() method, which prints "Starting the engine ...".

(2) return f"{self.name}'s electric engine started silently.";

- After calling the parent class's method, the return statement send to the caller

conceptional questions

PAGE NO.....
DATE / /

why we need super and when we need to override ~~in class~~ constructor in parent class using super()?

→ The constructor (--init-- method) is a special method in Python that is automatically called when you create an object of a class. It is used to initialize the object's attributes.

why need to override?

① Add new Attributes:

If the child class needs additional attributes that the parents class class doesn't have, you can override the constructor to initialize these new attributes.

Example

```
Class Car:
    def --init--(self, name):
        self.name = name # Parent class attributes

class ElectricCar(Car):
    def --init--(self, name, battery_capacity):
        super().--init--(name) # call the parent
                                 class's constructor
        self.battery_capacity = battery_capacity
                                 # new attributes

electric_car = ElectricCar("Tesla", 100)
print(electric_car.name) # output: Tesla
print(electric_car.battery_capacity) # output: 100.
```

PAGE NO.....
DATE / /

Here,

-> The ElectricCar class overrides the constructor to initialize the battery_capacity attribute

-> It calls the parent class's constructor to initialize the name attributes.

(3) Modify Parent Class's Attributes

```
Class Car:
    - - - - -
    - - - - -

class ElectricCar(Car):
    - - - - - - -
        super().__init__(name.upper())
                # Modify the name attributes
    - - - - - - - -

electric - - - - -
print - - - -
print - - -
```

(4) Prevent Direct Instantiation of Parent Class.

-> If the parent class is abstract (e.g. using the abc module), you can override the constructor in the child class to ensure the parent class cannot be instantiated directly.

```
from abc import ABC, abstractmethod

class Car(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def start_engine(self):
        pass

class ElectricCar(Car):
    def __init__(self, name, battery_capacity):
        super().__init__(name)
        self.battery_capacity = battery_capacity

    def start_engine(self):
        return f"{self.name}'s electric engine started silently."

electric_car = ElectricCar("Tesla", 100)
print(electric_car.start_engine())
# Output: Tesla's electric engine started silently.

# car = Car("Generic") # This will raise a TypeError.
```

Polymorphism

PAGE NO.....
DATE / /

→ Polymorphism means "many forms". In programming, it refers to the ability of different objects to respond to the same method call in their own unique way. Polymorphism allow you to write code that can work with objects of different types as long as they adhere to a common interface.

(1) Method Overriding

→ A child class provide its own implementation of a method that is already defined in the parent class

→ This allows the same method to behave differently in different classes.

Example of Abstract base classes (ABCs).

```
class Animal:
    @abstractmethod
    def sound(self):
        return "some generic sound"

class Dog(Animal):
    def sound(self): # override the sound
                     method
        return "woof!"

class Cat(Animal):
    def sound(self): # override the sound
                     method.
        return "Meow"

#create objects
animals = [Dog(), Cat()]  # list of objects
# call the sound method
for animal in animals:
    print(animal.sound())
```

PAGE NO.....
DATE / /

1. The object is in list form?

→ A list allows you to store multiple objects in a single variable.

→ You can the iterate over the list and perform the same operation (e.g. calling the sound() method) on each object.

(1)

↳ animals is a list containing a Dog object and a Cat object.

↳ Both Dog and Cat inherit from the Animal class, so they share a common method (sound())

Interface

1(4) Counter questions: What is object and what its datatypes then?

what is an object?

In python, everything is an object. An object is an instance of a class, and it represents a specific entity with: (1) Attributes (2) Methods.

what is the Data Type of an object?

→ The data type of an object is the class it belongs to.

Let say:

```
class Dog

my_dog = Dog() # create object

print(type(my_dog))

# <class '__main__.Dog'>
```

Here,
→ The data type of object [my-dog] is class [Dog], which is the class it belongs to.

There
(1) Everything is an Object:
→ In Python, everything (~~int~~ integers, strings, lists, functions, classes) is an object.
→ Built-in object like str, int etc.

(2) Attributes and Methods
→ object have attributes (data) and methods (behaviour).

(3) Data Type
→ The data type of an object is the class it belongs to.

(2) Method Overloading
→ A class has multiple methods with same name but different parameters.

→ Python does not support method overloading directly, but you can achieve similar behaviour using default arguments or variable-length arguments.

PAGE NO.
DATE / /

3) Duck Typing

-> In python, the type of an object is determined by its behavior (methods and properties) rather than its class.

-> If an object behaves like a duck (i.e. it has the required methods), it is treated as a duck.

```
Class Dog:
    def sound(self):
        return "Woof!"

Class Cat:
    def sound(self):
        return "Meow!"

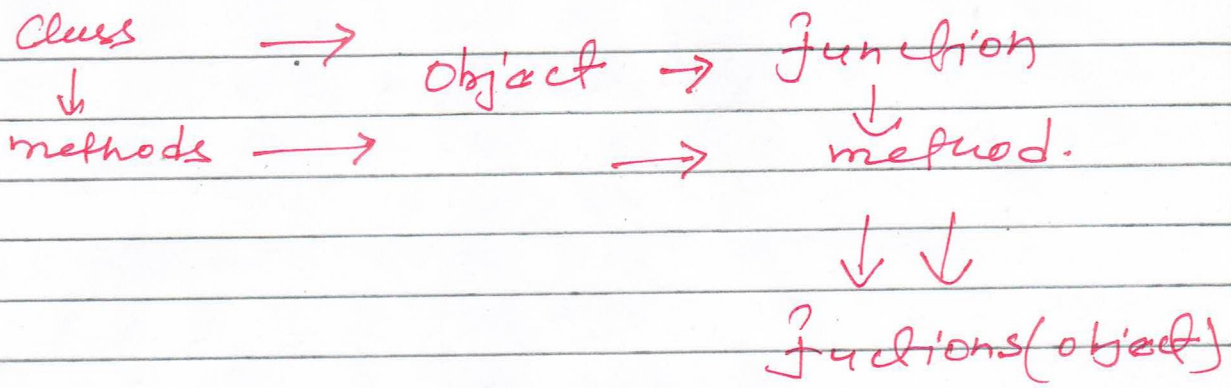
def make_sound(animal):  # Functions works with any object that has a sound()
    print(animal.sound())  method.

# Create objects
dog = Dog()
cat = Cat()

# Call the function
make_sound(dog)  # Output: Woof!
make_sound(cat)  # Output: Meow!
```

Here,

- The make_sound function works with any object that has a sound() method.
- This is an example of ~~ducking~~ typing: if it looks like a duck and quacks like a duck, it's a duck.



4) Abstract Base Classes (ABCs)

→ Abstract classes define a common interface that child classes must implement.

→ Polymorphism allows object of different classes to be treated as objects of the parent class.

5) Types of Polymorphism

(1) Compile-Time Polymorphism (Static Polymorphism)
→ achieved through method overloading.

(2) Run-Time Polymorphism (Dynamic Polymorphism)
→ achieved through method overriding.